

システム開発報告書

タクトーン

IMU ジェスチャで奏でる Arduino オーケストラ

情報工学科 2 年

25G1053

齋藤 翔太

チーム 23

片岡 聖 (24G1038) 地曳 賢人 (24G1075)

御代川 稜 (24G1131) 梅澤 颯太 (25G1021)

塩澤 匠生 (25G1065)

2026 年 7 月 10 日

表 1 主要要求と対応方針

要求	内容	対応方針
同期演奏	5 台構成で演奏タイミングを合わせる	指揮者ノードから楽器ノードへ UDP で拍情報を送る
可変テンポ 輪唱	指揮の速さに演奏を追従させる 複数パートがずれて同じ主旋律を演奏する	IMU の動きから拍とテンポを推定する 各パートに開始拍を設定し、同じ楽譜を時間差で使う
楽器音らしさ	目的の楽器に聞こえる音を鳴らす	Processing で倍音構造, ADSR, 原音サンプルを用いて音色を作る
エンターテインメント性	見て分かる演奏体験にする	指揮を振る動作そのものを演奏体験の中心に置く

1 はじめに

本報告書では、チーム 23 が制作した「タクトーン」について、システム全体の構成と筆者の担当範囲を述べる。本システムは、Arduino UNO R4 WiFi を用いて、指揮者役のマイコンと複数の楽器役マイコンを連携させ、PC 上の Processing で音を合成して演奏するシステムである。

授業資料では、同期信号の受信精度、タイミング誤差、エンターテインメント性、楽器音らしさなどが評価観点として示された。そこで本チームでは、単に決められたテンポで音を鳴らすのではなく、指揮者役マイコンの IMU から得られる動きを演奏制御に使う方針を採用した。この方針により、音楽経験が少ない人でも、指揮を振るという直感的な動作によって演奏へ参加できることを目指した。

筆者の主な担当は、楽譜データの作成、音階生成ロジックの検討、および Processing 上での 4 声輪唱確認用スケッチの作成である。本報告書は、現時点でリポジトリ内に存在する設計資料、作業ログ、Processing スケッチ、Arduino 向け楽譜データをもとにした初稿である。今後、発表時の実測値やチーム内での最終確認結果を加えて完成版にする。

2 要求と設計方針

本プロジェクトの目的は、外的要因に作用されず、機器さえあれば音楽未経験者でも演奏できる体験を実現することである。チーム内では、ユーザが指揮を振るテンポを変えたときに演奏速度も変わることを中心的なゴールとした。また、同期の成否を判断する指標として、楽器間の同期誤差 20 ms 以内を目安にした。

表 1 に、本システムで重視した要求と対応方針を示す。

表 2 Processing 確認用スケッチの主旋律パート

パート	音色	開始拍	音域設定
主旋律 1	フルート	0 拍	C5～A5
主旋律 2	トランペット	8 拍	C5～A5
主旋律 3	トロンボーン	16 拍	C3～A3
主旋律 4	オルガン	24 拍	C3～A3

3 システム全体構成

システムは、指揮者ノード、4 台の楽器ノード、PC 上の Processing アプリから構成される。指揮者ノードは Arduino UNO R4 WiFi の内蔵 IMU で動きを読み取り、拍、テンポ、強弱などに対応する制御情報を生成する。楽器ノードはそれぞれ自分の楽譜データを持ち、受信した拍情報に合わせて発音情報を PC へ送る。PC 側では Processing と Minim を用いて、受け取った発音情報または確認用スケッチ内の楽譜データから音を再生する。

チームの設計資料では、通信方式として UDP を採用している。UDP は到達保証を持たない一方で、低遅延で 1 対多の配信に向いている。本システムでは演奏タイミングが重要であるため、多少の欠損よりも遅延を小さくすることを優先した。また、Arduino 側の実装では、入力、ロジック、出力を分ける構成を採用し、将来的に 5 台のノードで共通化しやすい構造にする方針である。

4 個人担当範囲

筆者は、主に楽譜データと音色確認用 Processing スケッチを担当した。具体的には、「かえるのうた」を題材に、4 声の輪唱として成立するように開始拍、音域、音色、音量バランスを調整した。初期段階では金管 4 声を想定していたが、発表前確認用のスケッチでは、フルート、トランペット、トロンボーン、オルガンの 4 声構成に変更した。これにより、高音域の旋律、金管らしい中音域、低音域の支えを分けて確認できるようにした。

表 2 に、最終確認用 Processing スケッチにおける主旋律パートの構成を示す。4 パートは同じ基準譜面を使い、開始拍とオクターブ移調を変えることで輪唱を構成している。

5 実装内容

5.1 楽譜データ

楽譜データは、1 拍を 1 要素として扱う構造にした。Arduino 側では ScoreEvent 構造体を用意し、拍番号、MIDI ノート番号、ベロシティ、発音長、休符フラグを保持する。また、終盤の半拍音符を表現するために、subNote、subOffsetQ8、subDurationQ8 などの補助フィールドを用意している。

この設計により、1 拍単位の単純な旋律だけでなく、拍の途中に入る短い音も同じ配列内で扱える。「かえるのうた」の主旋律は 32 拍で構成し、Processing 確認用スケッチでは同じ MELODY_SCORE を 4 パートで共用した。開始拍を 0, 8, 16, 24 拍にずらすことで、輪唱として聞こえるようにした。

5.2 音色と再生確認

Processing 確認用スケッチでは、音色 JSON を読み込み、倍音比、倍音振幅、エンベロープをもとに音を合成する。金管系の音色では複数の正弦波を重ね、ADSR で音の立ち上がりと減衰を制御した。ドラムはキック、スネア、ハイハット、クラッシュを用意し、4 分の 4 拍子として 1・3 拍目にキック、2・4 拍目にスネアを配置した。各声部の入りでクラッシュとキックを重ねることで、輪唱の開始位置を聞き取りやすくした。

フルートについては、倍音合成だけでは自然な質感を出しにくかったため、tone_sample を含む音色 JSON を用いた。スケッチ側では ToneSampleUGen を実装し、原音サンプルをループ再生することで、倍音合成よりもフルートらしい音を得る方針にした。また、金管の立ち上がりがドラムに比べて遅く聞こえる問題に対し、拍位置は変えずに BRASS_ATTACK_SCALE = 0.45 としてエンベロープの attack を短縮した。

6 評価と考察

現時点で確認できていることは、Processing 上で 4 声輪唱とドラム伴奏を再生できることである。4 声は 0, 8, 16, 24 拍で順に入り、ドラムは 56 拍ぶん用意されているため、最後のパートが終わるまで伴奏を維持できる。また、MASTER_GAIN = 1.35, DRUM_AMPLITUDE = 0.095, FLUTE_SAMPLE_GAIN = 1.32 などを調整し、旋律とドラムの音量バランスを改善した。

一方で、システム全体としての同期誤差 20 ms 以内を満たしたかどうかは、本稿執筆時点では十分に記述できていない。これは、Processing 確認用スケッチが音色と楽譜の妥当性確認を目的としており、Arduino 5 台と PC を結合した実測評価とは役割が異なるためである。完成版の報告書では、実機での同期誤差、テンポ追従、発表環境での聴感確認の結果を追加する必要がある。

筆者の担当範囲に限ると、主な成果は、楽譜を共通化しながら開始拍と音域を変える構成にしたことである。この構成は、楽譜の修正箇所を減らし、4 声の入り方を比較しやすくする利点がある。また、音色 JSON をスケッチ内の data/フォルダに同梱したため、外部フォルダに依存せず再生確認できる。今後の課題は、Processing 上で調整した音色と音量を、実際の発表会場やスピーカ環境で確認し、必要に応じて再調整することである。

7 開発支援ツールの利用

本プロジェクトでは、設計資料の整理、Processing スケッチの修正方針検討、作業ログ作成の補助に生成 AI を利用した。筆者の担当では、4 声輪唱の開始拍、音域設定、ドラム伴奏、音量 balan

ス、README や作業ログの記述整理に利用した。

ただし、生成 AI の出力をそのまま採用したわけではない。Processing スケッチの定数、MELODY_PARTS、音色 JSON の配置、README の記述を照合し、実際にリポジトリ内に存在する内容と一致していることを確認した。また、作業ログでは、AI の提案内容、検証手順、未実施項目を分けて記録した。これにより、AI を使った部分と、人間が確認して判断した部分を区別できるようにした。

8 おわりに

本報告書では、チーム 23 の「タクトーン」について、システム全体の構成と筆者の担当した楽譜データ・Processing 確認用スケッチを中心に述べた。本システムは、指揮者ノードの IMU ジェスチャを使って演奏を制御し、複数の楽器ノードと PC 上の音色合成を組み合わせることで、指揮を振る体験を演奏に結び付けることを目指したものである。

筆者は、「かえるのうた」を 4 声輪唱として確認できる Processing スケッチを作成し、開始拍、音域、音色、ドラム伴奏、音量バランスを調整した。この成果により、Arduino 側へ反映する楽譜データと、発表時に聞こえる音色の確認を進めやすくなった。今後は、チーム内で実機の評価結果を整理し、同期誤差やテンポ追従の定量的な結果を追記することで、完成版の報告書に仕上げる。

参考文献

1. ハッカソン 1 授業資料, Hackathon1-260408 版.pdf, 2026 年.
2. ハッカソン 1 報告書評価資料, Hackathon1-Ruburic2026.pdf, 2026 年.
3. チーム 23 リポジトリ, プロジェクト概要, システム構成, 通信方針に関する設計資料, 2026 年.
4. チーム 23 リポジトリ, 齋藤翔太の week9~week11 作業ログおよび Processing スケッチ, 2026 年.

A 役割分担とスケジュール

表 A.3 に、設計資料に基づく主な役割分担を示す。

図 A.1 に、授業スケジュールをもとにした概略スケジュールを示す。

B プログラムソース

以下に、Processing 確認用スケッチにおけるパート定義の抜粋を示す。この定義により、同じ楽譜を開始拍とオクターブ移調だけ変えて 4 声輪唱として再生する。

表 A.3 主な役割分担

メンバー	主な担当
塩澤 匠生	Arduino 系プログラム全般, 指揮者ノード, 楽器ノード, 共通層
齋藤 翔太	楽譜データ作成, 音階生成ロジック検討, Processing 確認用スケッチ
梅澤 颯太	Processing による音色合成, 演奏再生
御代川 稜	議事録, 実機検証, 発表準備補助
地曳 賢人	議事録, 実機検証, 発表準備補助
片岡 聖	議事録, 実機検証, 発表準備補助

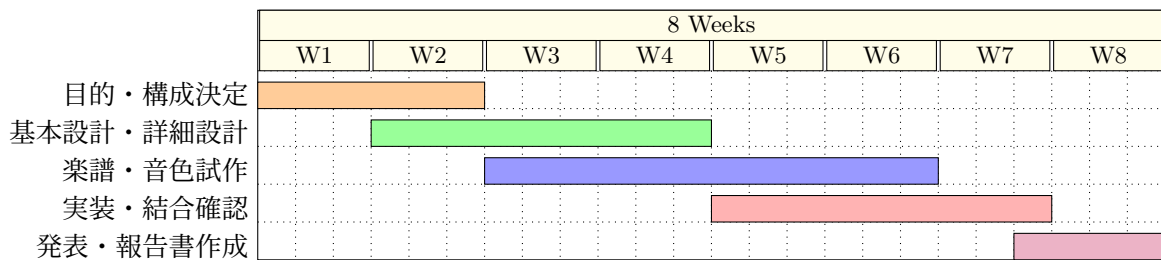


図 A.1 概略スケジュール

```

1  class PartDefinition {
2      String name;
3      String instrumentFile;
4      int startBeat;
5      int octaveShift;
6      float amplitude;
7
8      PartDefinition(String name, String instrumentFile,
9                      int startBeat, int octaveShift, float amplitude) {
10         this.name = name;
11         this.instrumentFile = instrumentFile;
12         this.startBeat = startBeat;
13         this.octaveShift = octaveShift;
14         this.amplitude = amplitude;
15     }
16 }
17
18 final PartDefinition[] MELODY_PARTS = {
19     new PartDefinition("主旋律1_フルート",
20                       "flute.tweaked.instrument.json", 0, 12, 0.28f),
21     new PartDefinition("主旋律2_トランペット",
22                       "trumpets.tweaked.instrument.json", 8, 12, 0.22f),
23     new PartDefinition("主旋律3_トロンボーン",
24                       "trombones.tweaked.instrument.json", 16, -12, 0.22f),
25     new PartDefinition("主旋律4_オルガン",
26                       "organ.tweaked.instrument.json", 24, -12, 0.25f)
27 };

```

次に, Arduino 側で用意した楽譜データ構造の抜粋を示す.

```

1  struct ScoreEvent {
2      uint16_t beatAt;           // パート開始からの拍番号
3      uint8_t noteNumber;       // MIDIノート番号。0は休符。
4      uint8_t velocity;         // 楽譜上の強さ。0～127。
5      uint16_t durationQ8;      // 1/256拍。256 = 1拍。
6      uint8_t flags;            // NoteOn / NoteOff / 休符。
7      uint8_t subNote;          // 拍内の第2音。0は未使用。
8      uint8_t subVelocity;      // subNoteの強さ。
9      uint16_t subOffsetQ8;     // 拍頭からのずれ。128 = 半拍。
10     uint16_t subDurationQ8;   // subNoteの発音長。
11 };

```